
Redes neuronales recurrentes (RNN)

Las redes neuronales recurrentes (RNN, *Recurrent neural networks*) son modelos creados para abordar problemas relacionados con el reconocimiento de patrones y se basan fundamentalmente en los mismos conceptos que los MLP (Perceptrones. multicapa) de propagación hacia adelante.

La diferencia radica en que, si bien los MLP, por definición, tienen múltiples capas, las RNN no las tienen, sino que tienen un ciclo dirigido a través del cual las entradas se transforman en salidas. Comenzaremos este documento abarcando varios modelos de RNN y lo finalizaremos con una aplicación práctica de las RNN.

Redes totalmente recurrentes

Imaginemos que tenemos una entrada, x , que introducimos en un modelo de RNN, donde definimos el estado como h , y las entradas se multiplican por una matriz de ponderaciones, W . Hasta ahora, todo es igual que en los modelos de redes neuronales descritos anteriormente, pero, como se mencionó anteriormente, las RNN realizan la misma tarea con las entradas a lo largo del tiempo.

Por ello, para calcular el estado actual de una red neuronal, derivamos la siguiente ecuación:

$$h_t = f(W_t x + W_R h_{t-1} + b), \text{ donde } f = \tanh, \text{ReLU}$$

$$y_t = f(W h_t b), \text{ donde } W_{t,r} = \text{pesos}, h_t = \text{capa oculta}, b = \text{sesgo}$$

La característica clave es que, cuando la red neuronal realiza estas operaciones, se despliega en múltiples estados nuevos, cada uno de los cuales depende de los estados anteriores. Dado que estas redes realizan la misma tarea para cada entrada, además de la dependencia funcional del modelo, a menudo se dice que las RNN tienen memoria. La figura 1 ilustra una RNN.

Esta forma de RNN se desarrolló en la década de 1980. Al igual que otras redes neuronales, múltiples capas de neuronas están conectadas por pesos, y cada peso se modifica mediante métodos de retropropagación. Modificamos nuestros pesos en función de una estadística de evaluación, que en este caso es la suma ponderada de las unidades de activación en un paso de tiempo determinado. El error total es la suma de todas estas sumas ponderadas individuales en todos los pasos de tiempo. Puede haber activaciones objetivo controladas por el experto para algunas de las unidades de salida en determinados intervalos de tiempo.

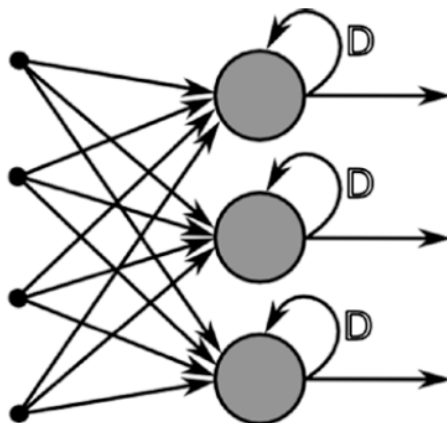


Figura 1: Arquitectura de una red neuronal recurrente.

Por ejemplo, si la secuencia de entrada es una señal de voz correspondiente a un dígito hablado, la salida objetivo final al final de la secuencia puede ser una etiqueta que clasifica el dígito. Para cada secuencia, su error es la suma de las desviaciones de todas las señales objetivo con respecto a las activaciones correspondientes calculadas por la red. Para un conjunto de entrenamiento de numerosas secuencias, el error total es la suma del error de todas las secuencias individuales.

Entrenamiento de RNN con retropropagación a través del tiempo (BPTT)

Sepp Hochreiter y Jurgen Schmidhuber, entre otros, son considerados entre los pioneros más importantes en el desarrollo de métodos de entrenamiento para el aprendizaje profundo. El método estándar se denomina **retropropagación a través del tiempo (BPTT, *Back-propagation through time*)**. BPTT es prácticamente igual que la retropropagación convencional, excepto que se creó para abordar un problema específico de las RNN: la evolución de un modelo a través de varios intervalos de tiempo.

Para cada época de entrenamiento, comenzamos entrenando con secuencias razonablemente pequeñas y aumentando gradualmente la longitud de dichas secuencias. Intuitivamente, esto se suele concebir como un entrenamiento con una secuencia de longitud 1, 2, hasta N , donde N es la longitud máxima posible de la secuencia. A continuación, se presenta una ecuación que describe este fenómeno de forma más sucinta

$$\delta_{p,j}(t-1) = \sum_h^m \delta_{ph}(t) u_{hj} f'(s_{pj}(t-1))$$

donde t = paso de tiempo, h = índice de nodos ocultos en t , j = paso de nodos ocultos en $t = 1$, y δ = errores. En detalle, podemos visualizar el fenómeno de la siguiente manera: definimos W como la matriz de pesos de la capa de salida con la ecuación

$$W(t+1) = W(t) + \eta s(t) \mathbf{e}_o(t)^T$$

donde $\mathbf{e}_o$ = errores de la capa de salida:

$$\mathbf{e}_o(t) = \mathbf{d}(t) - \mathbf{y}(t)$$

Ahora contamos con k secuencias, mediante las cuales desplegamos la red en una red de propagación hacia adelante regular, como hemos observado hasta ahora. Sin embargo, la capa recurrente del modelo RNN recibe simultáneamente la entrada de la capa anterior y de la siguiente. Para compensar el cambio en los pesos que se produce por la entrada simultánea al retropropagar los errores, promediamos las actualizaciones que recibe cada capa.

Redes neuronales Elman

Las arquitecturas RNN recibieron contribuciones adicionales de Jeffrey Elman, a quien se le atribuye la creación del modelo de red Elman que lleva su nombre. Principalmente, las arquitecturas que Elman construyó fueron para algoritmos de procesamiento del lenguaje, pero también pueden ser útiles para cualquier problema en el que los datos de entrada sean secuenciales o basados en series temporales. La figura 2 muestra la estructura básica de una red Elman.

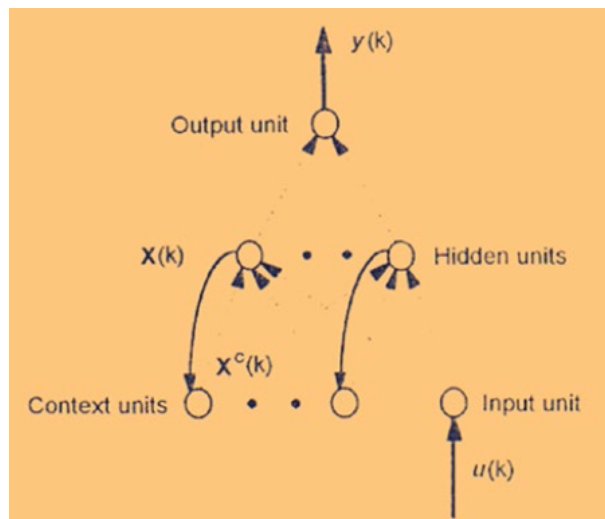


Figura 2: Ilustración de la red Elman.

Elman incluyó una capa de unidades de contexto en esta arquitectura que se distinguen por su funcionalidad, que se centra en gran medida en los estados internos previos. Una de las

distinciones clave de una red Elman es que la salida de la capa oculta también alimenta las unidades de contexto de la capa anterior. Sin embargo, los pesos que conectan las unidades de contexto y la capa oculta tienen un valor constante de 1, lo que hace que la relación sea lineal.

Posteriormente, la capa de entrada y la capa de contexto activan simultáneamente la capa oculta, tras lo cual esta también genera un valor al realizar el paso de actualización. Durante la siguiente época, la secuencia de entrenamiento descrita anteriormente se repite, excepto que aquí observamos que la capa con las unidades de contexto adopta los valores de la capa oculta de la época anterior. Esta característica de las unidades de contexto se ha descrito coloquialmente como la red con memoria. El entrenamiento de esta red neuronal requiere numerosos pasos, cuyo número depende en última instancia de la longitud de la cadena elegida.

Compresor de historia neuronal

El gradiente de desaparición (*vanishing gradient*) se refiere específicamente a que los gradientes en las capas anteriores de una red se vuelven infinitesimalmente pequeños. Esto ocurre debido a la función de activación que utilizamos, generalmente una tangente o una sigmoidea. Dado que estas funciones de activación comprimen las entradas en rangos relativamente pequeños para facilitar la interpolación de los resultados, la derivación de los gradientes se vuelve significativamente más difícil.

Si se repite este proceso de comprimir las entradas después de varias capas apiladas, para cuando se retropropaga a la primera capa, el gradiente se habrá *desvanecido*.

El problema de la desaparición de los gradientes se solucionó parcialmente mediante la creación de compresores de historial neuronal, un modelo generativo temprano implementado como una pila no supervisada de redes neuronales recurrentes.

El nivel de entrada aprende a predecir su siguiente entrada a partir del historial de entradas anterior. En la RNN de nivel superior, las entradas se componen únicamente de las entradas impredecibles de un subconjunto de las RNN de la pila, lo que garantiza que el estado interno se recalcule con poca frecuencia. De este modo, cada RNN de alto nivel aprende una representación comprimida de la información de la RNN inferior.

Por diseño, podemos reconstruir con precisión la secuencia de entrada a partir de la representación de la secuencia en el nivel superior. Cuando utilizamos datos secuenciales con considerable predictibilidad, el aprendizaje supervisado puede utilizarse para clasificar secuencias considerablemente más profundas a través de la RNN de más alto nivel.

Memoria a largo plazo (LSTM)

La LSTM (*Long short-term memory*) es un modelo cada vez más popular, cuya fortaleza reside en la gestión de huecos de tamaño desconocido entre señales en el ruido de los datos.

Desarrolladas a finales de la década de 1990 por Sepp Hochreiter y Jurgen Schmidhuber, las LSTM son universales, de modo que, cuando hay suficientes unidades de red, cualquier cálculo que una computadora pueda realizar puede replicarse con ellas, suponiendo que se disponga de una matriz de ponderaciones correctamente calibrada. La figura 3 la ilustra.

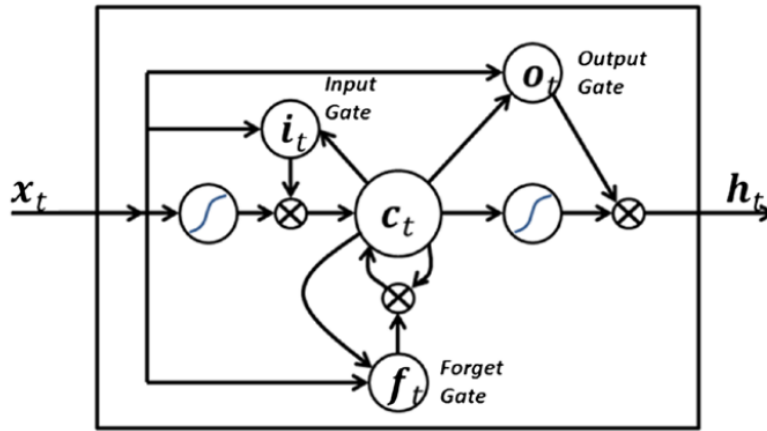


Figura 3: Visualización de la red de memoria a largo plazo.

La variedad de aplicaciones de las LSTM explica en parte su popularidad, ya que se utilizan a menudo en los campos del control de robots, la predicción de series temporales, el reconocimiento de voz y otras tareas. A diferencia de las unidades que solemos ver en otras arquitecturas de RNN, las redes LSTM contienen bloques. Otro factor clave que distingue a los LSTM es su capacidad de recordar un valor dado durante largos periodos de tiempo y las puertas dentro del modelo que determinan varios atributos de la secuencia de entrada.

Entre las consideraciones de las puertas se encuentran la significancia de la entrada, cuándo se debe conservar la memoria o realizar la recolección de basura y eliminar los datos, y el tiempo del valor de salida. Una implementación típica de un bloque LSTM se muestra en la figura 3. Las unidades sigmoideas en un LSTM estándar contienen la ecuación

$$y = s \left(\sum_{i=1}^N w_i x_i \right),$$

donde s es una función de aplastamiento (en muchos casos, a menudo una función logística o cualquier función de activación, como se describió en modelos anteriores). En la figura 3,

la unidad sigmoidea situada más a la izquierda alimenta la entrada a la “memoria” del bloque LSTM. A partir de este punto, las demás unidades de la figura actúan como puertas, que permiten o impiden el acceso a la memoria de la LSTM. La unidad i , que designamos como la puerta de entrada del diagrama, bloqueará la entrada a la memoria de todos los valores muy pequeños (cerca de cero).

La puerta de olvido (*forget gate*), la unidad situada en la parte inferior de la figura, “olvida” el valor que estaba recordando y lo descarta de la memoria. La unidad situada en la esquina superior derecha de la figura es la “puerta de salida”, que determina si el valor almacenado en la memoria de la LSTM debe emitirse. Ocasionalmente, observamos unidades representadas por los siguientes símbolos: Π o Σ . Las unidades con el símbolo de suma se reintroducen en el bloque LSTM para facilitar la memorización del mismo valor a lo largo de varios intervalos de tiempo sin que se deteriore.

Normalmente, este valor también se introduce en las unidades de tres puertas para mejorar sus procesos de toma de decisiones. El producto de Harnard, o producto de entrada de matrices utilizado en LSTM, se expresa de la siguiente manera en notación de índice:

$$(A \circ B)_{i,j} = A_{i,j} \times B_{i,j}$$

LSTM tradicional

Arriba, se muestran las capas de una LSTM por las que pasan los datos.

$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f),$$

$$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i),$$

$$o_t = \sigma_g(W_o x_t + U_o h_{t-1} + b_o),$$

$$c_t = f_t \circ c_{t-1} + i_t \circ \sigma_c(W_c x_t + U_c h_{t-1} + b_c),$$

$$h_t = o_t \circ \sigma_h(c_t),$$

donde x = vector de entrada, h_t = vector de salida, ct = estado de la celda, (W, U, b) = matrices y vector de parámetros, $(f_t, i_t$ y $o_t)$ = información recordada, información adquirida y salida,

respectivamente, σ_g = función sigmoidea, σ_c = tangente hiperbólica original, σ_h = tangente hiperbólica original.

LSTM de entrenamiento

BPTT se utiliza para LSTM, pero debido a las características especiales de la LSTM, también podemos utilizar el descenso de gradiente mediante BP como se haría tradicionalmente. Los gradientes evanescentes en los LSTM se gestionan específicamente mediante el carrusel de errores. Las LSTM “recuerdan” sus errores retropropagados, que luego se retroalimentan a cada peso. Por lo tanto, la retropropagación regular es eficaz para entrenar un bloque LSTM y que recuerde valores durante periodos muy largos.

Amortiguamiento estructural dentro de las RNN

Si utilizamos un método de gradiente conjugado y este se desvía demasiado de la x original, la estimación de la curvatura se vuelve inexacta y podríamos observar una incapacidad para converger al óptimo global. Según Martens y Sutskever, se recomienda el *amortiguamiento estructural* (structural dampening) al utilizar métodos de gradiente conjugado. Con este método, penalizamos las grandes desviaciones de x , donde la fórmula viene dada por

$$\tilde{f}_d(x + \Delta x) = \tilde{f}(x + \Delta x) + \lambda \|\Delta x\|^2,$$

donde $\|\Delta x\|^2$ es la magnitud de la desviación. λ , similar a la regresión de cresta, sirve como parámetro de ajuste.

El parámetro de ajuste es adaptativo y se selecciona mediante un proceso similar al del algoritmo de Levenburg-Marq descrito en el Capítulo 3. Se sugiere encontrar una razón de reducción, dada por la siguiente ecuación:

$$\rho = \frac{f(x + \Delta x) - f(x)}{\tilde{f}(x + \Delta x) - \tilde{f}(x)}$$

Algoritmo de actualización de parámetros de ajuste

Los pesos se actualizan en cada paso de tiempo y, por lo tanto, aumentar el valor de esta matriz puede causar cambios drásticos en el resultado:

$$\text{If} \left(\rho > \frac{3}{4} \right) \{$$

$$\lambda \rightarrow \frac{2}{3}\lambda.$$

} Else If $\left(\rho < \frac{1}{4}\right)$ {

$$\lambda \rightarrow \frac{3}{2}\lambda$$

} Else If $\left(\frac{1}{4} < \rho < \frac{3}{4}\right)$ {

$\lambda \rightarrow \lambda$, (no se actualiza)}

Ejemplo práctico de RNN - Detección de patrones

Tomemos como ejemplo la predicción de series temporales basadas en datos secuenciales. En este caso, intentaremos predecir la producción de leche en diferentes épocas del año (Figuras 4 y 5). Comencemos examinando nuestros datos para comprenderlos:

```
> #Clear the workspace
> rm(list = ls())
> #Load the necessary packages
> library(rnn)
> #Function to be used later
> #Creating Training and Test Data Set
> dataset <- function(data){
+   x <- y <- c()
+   for (i in 1:(nrow(data)-2)){
+     x <- append(x, data[i, 2])
+     y <- append(y, data[i+1, 2])
+   }
+   #Creating New DataFrame
+   output <- cbind(x,y)
+   return(output[1:nrow(output)-1,])
+ }
```

Al trabajar con datos de series temporales, tendremos que realizar una cantidad considerable de transformaciones de datos. En particular, debemos crear variables X e Y ligeramente diferentes de los datos proporcionados. Desde la función dataset(), creamos una nueva variable X, que corresponde al paso de tiempo t, a partir de la variable Y original. Creamos una nueva variable Y que es t + 1 a partir de la variable Y original. Luego, truncamos los

datos en una fila para eliminar la observación faltante. A continuación, cargamos y visualizamos los datos (mostrados en las Figuras 4 y 5):

```
> #Monthly Milk Production: Pounds Per Cow
> data <- read.table("monthly-milk-production-pounds-p.csv", header = TRUE, sep = ",")
> #Plotting Sequence
> plot(data[,2], main = "Monthly Milk Production in Pounds", xlab = "Month",
+       ylab = "Pounds",
+       lwd = 1.5, col = "cadetblue", type = "l")
> #Plotting Histogram
> hist(data[,2], main = "Histogram of Monthly Milk Production in Pounds",
+       xlab = "Pounds", col = "red")
```

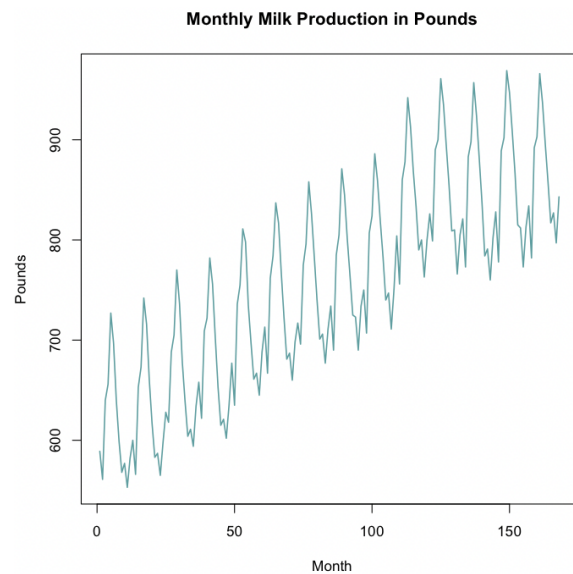


Figura 4: Visualización de la secuencia.

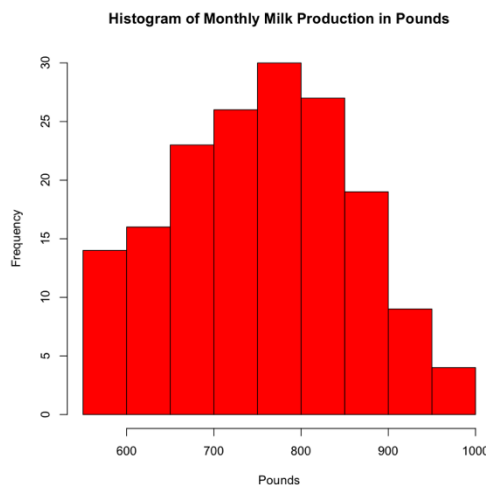


Figura 5: Visualización de los datos de leche mediante histograma.

Como puedes observar, nuestros datos presentan una marcada desviación a la derecha con respecto a la frecuencia de los valores, a pesar del aparente amplio rango de valores.

Ahora que has comprendido visualmente nuestros datos, procedamos a prepararlos para su introducción en la RNN:

```
> #Creating Test and Training Sets
> newData <- dataset(data = data)

> #Creating Test and Train Data
> rows <- sample(1:120, 120)
> trainingData <- scale(newData[rows, ])
> testData <- scale(newData[-rows, ])
```

Recomiendo a todos los usuarios que utilicen el **escalado máximo-mínimo** antes de introducir sus datos en una RNN, ya que ayuda significativamente a reducir los errores de una red neuronal. Al igual que la normalización estándar, el escalado máximo-mínimo reduce significativamente el rango de los datos de entrada, pero lo hace clasificando las observaciones entre 0 y 1 en lugar de indicar cuántas desviaciones estándar se encuentran respecto a la media de los datos. Una vez realizado este paso, podemos introducir los datos. Los usuarios pueden experimentar con los parámetros, pero he entrenado la red para un buen rendimiento. Ahora, evaluemos los resultados de entrenamiento y prueba (mostrados en las Figuras 6 y 7):

```
> #Max-Min Scaling
> x <- trainingData[,1]
> y <- trainingData[,2]
> train_x <- (x - min(x))/(max(x)-min(x))
> train_y <- (y - min(y))/(max(y)-min(y))
> #RNN Model
> RNN <- trainr(Y = as.matrix(train_x), X = as.matrix(train_y),
+   learningrate = 0.04, momentum = 0.1,
+   network_type = "rnn", numepochs = 700, hidden_dim = 3)
```

Cuidado en esta parte, tardará según la velocidad de tu computadora.

```
> y_h <- predictr(RNN, as.matrix(train_x))
> #Comparing Plots of Predicted Curve vs Actual Curve: Training Data
> plot(train_y, col = "blue", type = "l", main = "Actual vs Predicted Curve", lwd = 2)
> lines(y_h, type = "l", col = "red", lwd = 1)
> library(Metrics)
> cat("Train MSE: ", mse(y_h, train_y))
Train MSE: 0.01251946
```

```

> #Test Data
> testData <- scale(newData[-rows, ])
> x <- testData[,1]
> y <- testData[,2]
> test_x <- (x - min(x))/(max(x)-min(x))
> test_y <- (y - min(y))/(max(y)-min(y))
> y_h2 <- predictr(RNN, as.matrix(x))
> #Comparing Plots of Predicted Curve vs Actual Curve: Test Data
> plot(test_y, col = "blue", type = "l", main = "Actual vs Predicted Curve", lwd = 2)
> lines(y_h2, type = "l", col = "red", lwd = 1)
> cat("Test MSE: ", mse(y_h2, test_y))
Test MSE: 0.06718617

```

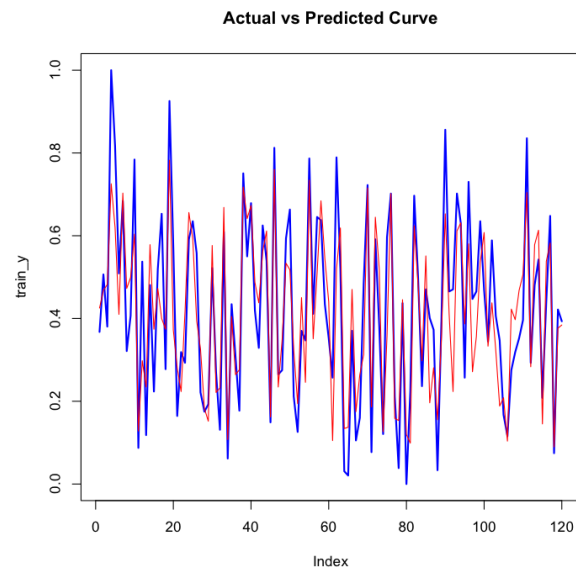


Figura 6: Rendimiento de los datos de entrenamiento.

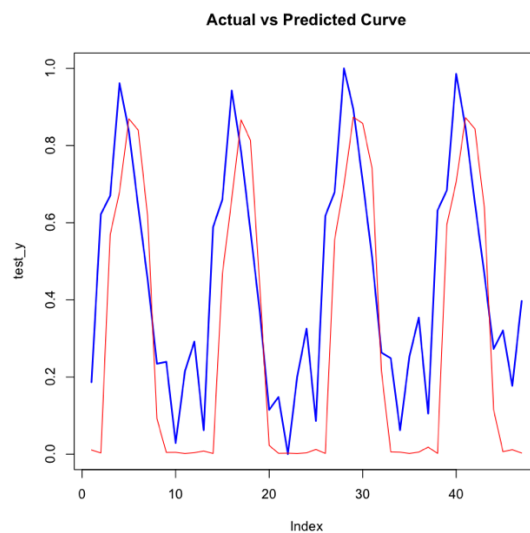


Figura 7: Rendimiento del conjunto de prueba.

Respectivamente, los conjuntos de entrenamiento y prueba tienen MSE de 0.01251946 y 0.06718617. Si bien el MSE del conjunto de entrenamiento es menor, probablemente se deba a que este es significativamente mayor que el de prueba.

Podemos observar que el rendimiento de la prueba es menos preciso que el del conjunto de entrenamiento al comparar visualmente las curvas en los respectivos gráficos. Como puedes observar, la curva real, tanto en el conjunto de entrenamiento como en el de prueba, presenta una varianza mayor de la que la RNN puede capturar por completo. Si está leyendo el documento, la curva real está en azul y la curva predicha está en rojo.

Resumen

Este documento ha abordado eficazmente los ejemplos de RNN más frecuentes. También ha guiado al lector en la resolución de problemas con datos de series temporales.